Introduction to Machine Learning

Create/join a group

1. One person (and only one) should create a group.
2. Everyone else should enter the group name below (in the form `groupname_0000`).

   Join group: Submit

To join another group or leave your current group, reload the page.

You are not in a group.

Neural Networks II

Due: Wednesday, October 14, 2026 at 04:15 PM

This lab builds on the material in the lecture and course notes on Neural Networks.

1) Exploring neural networks

Notes on the functions in the code boxes below:

- The initialization of weights in a neural network is random, so subsequent runs might produce different results.
- We are training using stochastic gradient descent.
- `iters` indicates how many single steps of SGD to run, each using one training point.
- Pay attention to the decision boundaries, shown in the answers after you run the code. Also note that the accuracy of the network on the training data is printed above the graph.

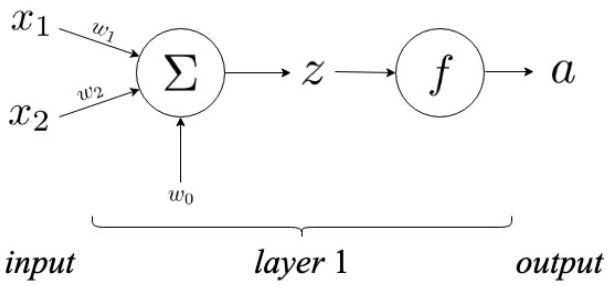
The code we are running is what you'll implement in the released homework!

1.1) Simple separable dataset

Here's a simple data set of four data points.

```
X = np.array([[2, 5], [3, 2], [9, 6], [12, 5]])
Y = np.array([[1],[0],[1],[0]])
```

The code below runs a simple neural network composed of a single sigmoid unit with two inputs, using negative log likelihood (NLL) loss:



The function `sigmoid_nn` in the code box below plots the classifier found after training for the specified number of iterations (`iters`) with the specified learning rate or step size (`lr`). The classifier predicts label +1 if the output of the sigmoid, a, is greater than 0.5 and label 0 otherwise--remember that the range of the sigmoid function is $(0, 1)$.

```
def run():
    return sigmoid_nn(iters=100, lr=0.05)

```

Run Code | Submit | Ask for Help **100.00%**

You have infinitely many submissions remaining.

1.1.1)

What is the shape of the classifier's decision boundary produced by this network? Explain.

1.1.2)

What combination of `iters` and learning rate `lr` can you use to get a relatively consistent 100% accuracy? What trends do you notice in the iteration vs loss and accuracy vs loss graphs as you increase learning rate (e.g. `lr` = 0.001, .01, and .1)?

Note that there is randomness in the outcome due to randomly initialized weights and the random order of data selected by SGD.

1.2) Not-XOR

The XOR dataset is a classic example showing the need for more than one layer of non-linear units.

Here, we will consider a function outputting the **negation** of XOR, **Not-XOR**, where the training dataset is:

```
X = np.array([[0, 0], [1, 1], [0, 1], [1, 0]])
Y = np.array([[1], [1], [0], [0]])
```

1.2.1)

Draw a plot showing the locations of these data points in x_1, x_2 coordinate space, with the corresponding labels. Is this

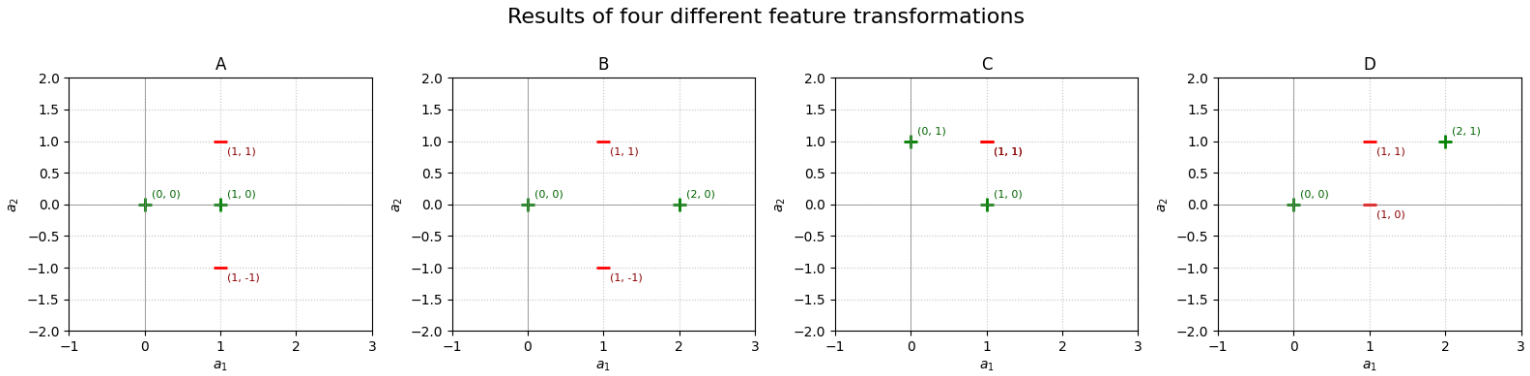
dataset linearly separable?

1.2.2)

Inspired by the material from the previous week, we can try to find a feature transformation of x_1 and x_2 such that the dataset is linearly separable in the transformed feature space. We try four different feature transformations. The resulting transformed datasets (plotted in the **transformed** *feature space*) are shown below, where a_1 and a_2 are the transformed features. The green '$+$' represent the transformed positive data points, and the red '$-$' represent the transformed negative data points.

Match the feature transformations to their corresponding plot. Which of them result in a dataset that is linearly separable?

Note: Boolean expressions that evaluate to `True` *are assigned a value of* 1*, and* 0 *if they evaluate to* `False`*.*



As an example, verify for yourself that the transformation $a_1 = x_1 + x_2,\ a_2 = x_1 - x_2$ corresponds to plot B.

Which plot corresponds to $a_1 = x_1 + x_2,\ a_2 = (x_2 > 0)$? --

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

Which plot corresponds to $a_1 = (x_1 > 0 \text{ OR } x_2 > 0),\ a_2 = x_1 - x_2$ --

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

Which plot corresponds to $a_1 = (x_1 > 0 \text{ OR } x_2 > 0),\ a_2 = (x_1 < 1 \text{ OR } x_2 < 1)$? --

Submit View Answer Ask for Help

You have infinitely many submissions remaining.

1.2.3)

Can we find a_1 and a_2 which are obtained from a *linear* transformation of x_1 and x_2 that yield a separable dataset?

1.2.4)

We will proceed with $a_1 = (x_1 > 0 \text{ AND } x_2 > 0)$, and $a_2 = (x_1 < 1 \text{ AND } x_2 < 1)$. These correspond to AND and NOR logical operations. Sketch a plot of the transformed dataset under this feature transformation. Is it linearly separable? If so, draw a possible linear classifier.

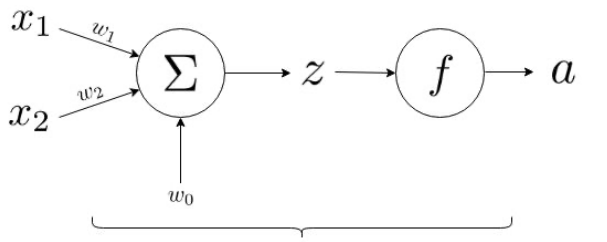
Note: $a_2 = (x_1 < 1 \text{ AND } x_2 < 1)$ corresponds to the NOR operator because by De Morgan's Laws , $(\text{NOT } x_1) \text{ AND } (\text{NOT } x_2)$ is equivalent to $\text{NOT } (x_1 \text{ OR } x_2)$

1.2.5)

We would hope that our neural network can replicate this cleverly hand-designed feature transformation. Let's see how a single unit could approximate logical operations. Consider the following truth table representing the logical AND, OR, and NOR operators. We would like to represent each of these functions as a neural network (single unit), using the same structure as in Problem 1.1. Find weights w_0, w_1 and w_2 that represent the (i) AND operation, (ii) OR operation and (iii) NOR operation on binary inputs x_1 and x_2.

As a reminder, $f(z) = \sigma(z)$.

Set the weights such that the output (a) corresponding to the true (1) label is larger than 0.99 and the output corresponding to the false (0) label is smaller than 0.01. Here the thresholds 0.99 and 0.01 are arbitrarily set for answer-checking reasons.



x_1	x_2	AND	OR	NOR
0	0	0	0	1
0	1	0	1	0
1	0	0	1	0
1	1	1	1	0

Hint: Think about the magnitude of the weights needed to make the output labels larger than 0.99 or smaller than 0.01.

Enter a list of values for $[w_0, w_1, w_2]$ that represent the **AND** operation:

Check Formatting | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

Enter a list of values for $[w_0, w_1, w_2]$ that represent the **OR** operation:

Check Formatting | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

Enter a list of values for $[w_0, w_1, w_2]$ that represent the **NOR** operation. Hint: think about the sign of the weights.

Check Formatting | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

1.2.6)

Think about the linear separator you came up with in 1.2.4. What Boolean operation could it represent on a_1 and a_2? In particular, combining everything, which of the Boolean expressions below are equivalent to NOT-XOR?

Hint 1: You will find it helpful to write out the truth table for Not-XOR and possibly XOR.

Hint 2: By De Morgan's Laws, $(\text{NOT } x_1 \text{ AND NOT } x_2)$ is equivalent to $\text{NOT } (x_1 \text{ OR } x_2)$

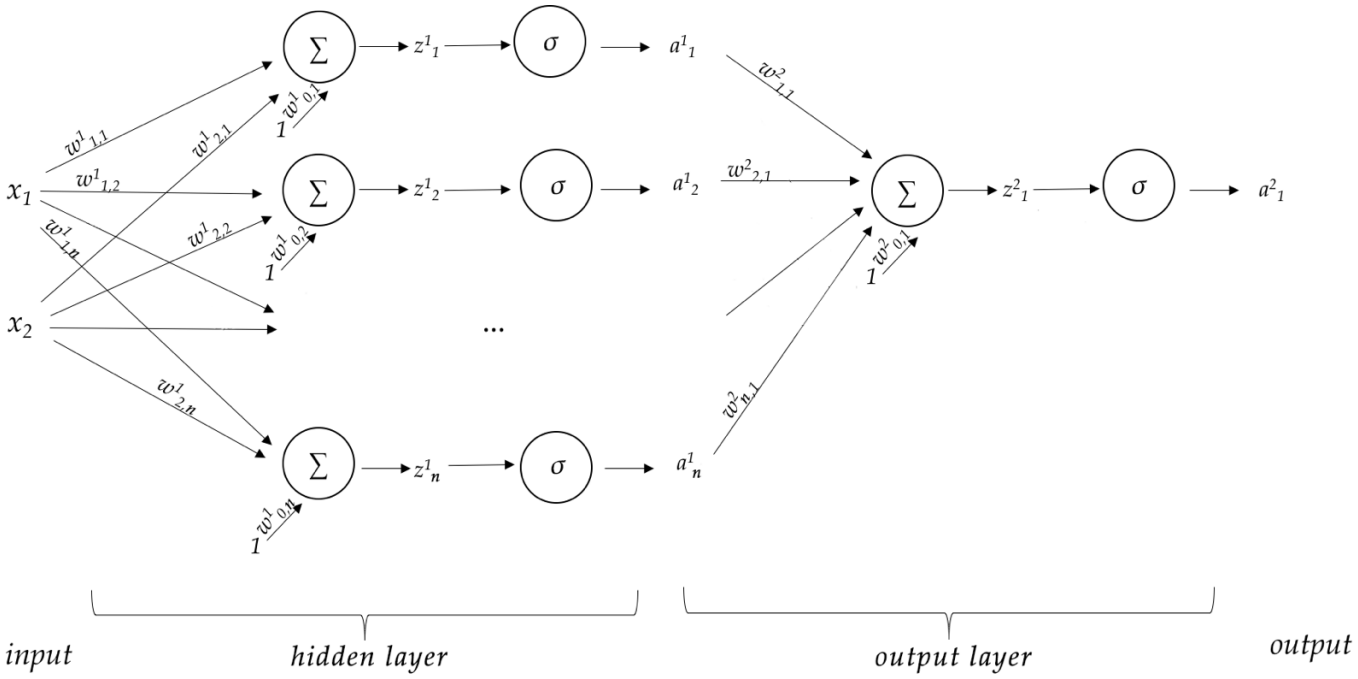
- (x_1 AND x_2) AND (NOT x_1 AND NOT x_2)
- (x_1 OR x_2) AND (NOT x_1 OR NOT x_2)
- (x_1 AND x_2) OR (NOT x_1 AND NOT x_2)
- (x_1 OR x_2) OR (NOT x_1 OR NOT x_2)

Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

1.2.7)

Consider the following network with one hidden layer of Sigmoid (σ) units:



We can use this network to closely approximate the Not-XOR function as constructed in 1.2.6 above. What is the minimal number of units we need in the hidden layer (what is n in the figure)? Be prepared to explain your answer during checkoff.

Hint: What you did in earlier parts of 1.2 will be helpful here.

Enter the minimal number of units we need in the hidden layer:

Check Formatting | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

Checkoff 1:
Have a check-off conversation with a staff member, to explain your answers.

Ask for Help | Ask for Checkoff

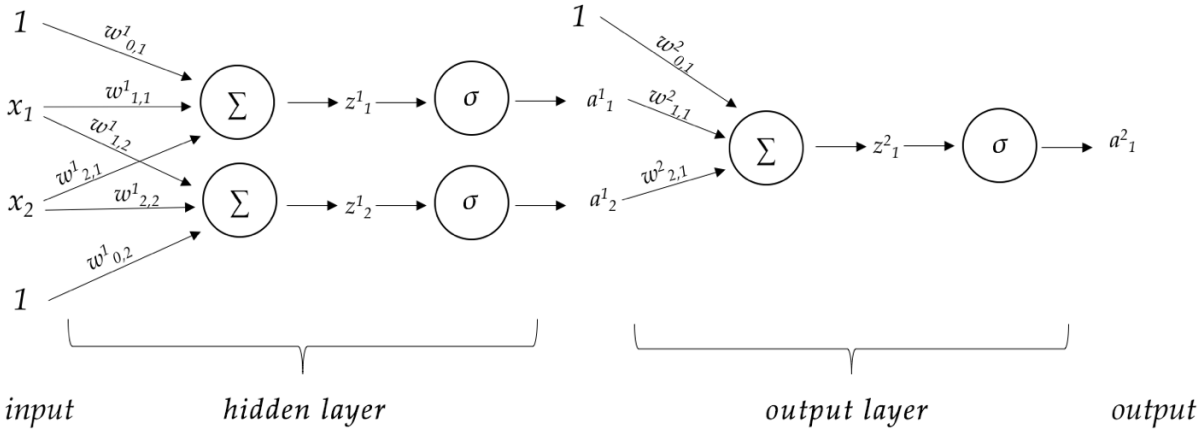
Your checkoff is for Section 3, 1pm-2:30pm, 34-501.
If you're in a different section, be sure to switch your section.

1.2.8)

Now that we have our architecture, find the weights and offsets so the network represents Not-XOR.

We expect you to use the minimum number of hidden units found in 1.2.7.

x_1	x_2	a_1^1	a_2^1	a_1^2
0	0			1
0	1			0
1	0			0
1	1			1



The checker below will help you verify your answer. It takes in the weights/offset based on some syntax. For example, if you use one hidden unit, the format should be:

$$[[\underbrace{w^1_{0,1}, w^1_{1,1}, w^1_{2,1}}_{\text{weights to the hidden unit}}], \underbrace{[w^2_{0,1}, w^2_{1,1}]}_{\text{weights to the output unit}}]$$

If you use two hidden units:

$$[[\underbrace{w^1_{0,1}, w^1_{1,1}, w^1_{2,1}}_{\text{weights to the 1st hidden unit}}, \underbrace{w^1_{0,2}, w^1_{1,2}, w^1_{2,2}}_{\text{weights to the 2nd hidden unit}}], [w^2_{0,1}, w^2_{1,1}, w^2_{2,1}]]$$

where $w^1_{0,1}, w^1_{0,2}, w^1_{0,n}, \ldots, w^2_{0,1}$ are offsets of the hidden units and output unit.

Explain how the network works with reference to your earlier drawing of the Not-XOR data in x_1 and x_2 space.

Hint 1: Reuse **AND**, **OR**, and **NOR** networks from problem 1.2.5 as components of a larger network.

Hint 2: Make your weights larger ($|w| \geq 5$) and think about how increasing the magnitude of the weights affects the output of a **sigmoid** unit.

Hint 3: Check your weights for the correct logic table for Not-XOR. Think of how we created feature transformations for a_1 and a_2 in 1.2.4.

Try to spend just 10 minutes on this; if you are still stuck, join the queue and chat with a staff member.

Enter a list of lists for weights that represent Not-XOR operation:

Check Formatting | Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

1.2.9)

The `single_hidden_layer_nn` function in the code below runs on the dataset using a network with a single hidden layer of sigmoid units, a sigmoid output layer, and NLL loss.

Try playing around with the **number of hidden units**, the **number of iterations**, and the **learning rate**.

First, try to see if you can get a network of the size you found above to separate the data reliably (try several times in a row).

Now try experimenting with larger networks. How many hidden units did you need to get good results? What happened when you changed each hyperparameter? Discuss what you think might be going on during your checkoff.

Note: After you run the code below, if the plot shows up as all one color, it means that the separator does not intersect the region we are plotting. If that happens, try increasing `hidden_units` and `iters`.

```
def run():
    return single_hidden_layer_nn(hidden_units=1, iters=1000, lr=0.05)

```

Run Code | Submit | Ask for Help

You have infinitely many submissions remaining.

1.3) Hard data set

In this next example, we use a much harder data set that is **barely** separable (although not linearly). We'll try a two-layer network architecture.

1.3.1)

We consider a network that has two hidden layers of ReLU units, and a sigmoid output unit with NLL loss. In the `two_hidden_layer_nn` function call below, you can play around with the number of hidden units in the hidden layers (the same number is used in both hidden layers), the number of iterations, and the learning rate, in trying to separate our harder dataset.

Experimenting with different parameters in the code below, can you get this architecture to separate the dataset with at least 95% accuracy? How about 100% accuracy (don't spend too long looking for 100%)? If you can't, explain why not. If you can, explain why. Make sure your accuracy is reliable by running the code several times.

```
def run():
    return two_hidden_layer_nn(hidden_units=2, iters=500, lr=0.05)

```

Run Code | Submit | Ask for Help

You have infinitely many submissions remaining.

1.3.2)

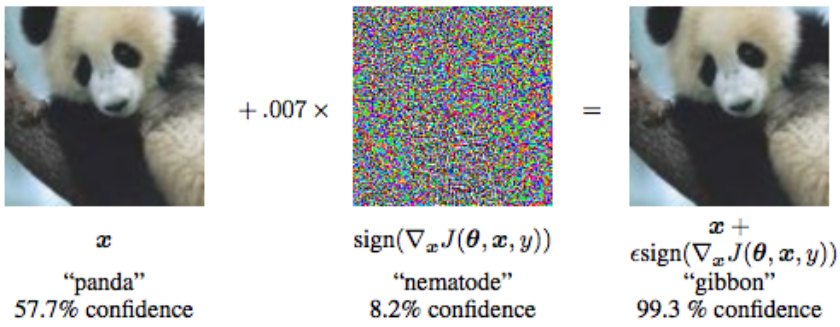
Do you think it's a good idea to try to find a "perfect" separator for this data? Explain.

2) Adversarially Robust Models

Now we'll explore potential limitations in the generalization abilities of neural networks (which also are a concern in other machine-learning mechanisms) via an idea called "adversarial examples."

The idea is this: once we have trained a neural network to perform well on training and even test data, we would hope that the network will produce similar results on new unseen data. For inputs "similar" to those we trained with, we would hope that outputs (like classification decisions) would also be similar to those for our training examples. But what if someone (an adversary) is trying to trick the network into making a wrong classification decision for a "similar" input?

These modified inputs designed to trick machine learning models exist for text, images, and even real-world objects:



(Ian Goodfellow et al., 2014.)

In the example above, they start with an initialized panda image x, and use the sign of the gradient of the loss $J(\theta, x, y)$ to perturb the input away from the "panda" class.

We will explore this idea first with a very simple example to build intuition. We will consider trying to find an adversarial example that "looks" (to humans, for example) like a class 1 training example, but that the network wrongly classifies as class 0 (or vice versa).

2.1) Gradient descent on the input

How do we find an adversarial example?

In normal neural-network training, we use gradient descent to adjust the weights to minimize a loss function. Now, to find an example that fools our network, we are going to **freeze the weights of the network, starting from some correctly classified input $x^{(0)}$, and use gradient info to try to find a new (hopefully nearby) input x that is misclassified by the network**.

In this part, we'll consider a linear logistic classifier, $h(x) = \sigma(\theta^T x + \theta_0)$. Recall we interpret the output of $h(x)$ as the probability (or confidence) that example x is assigned to class 1. Correspondingly, $1 - h(x)$ is the probability that x is assigned to class 0.

Using this interpretation of the classifier output $h(x)$, our strategy for finding an adversarial example will be to start with an example that $h(x)$ classifies correctly as class 0, and use **gradient ascent** on x to **increase** the value of $h(x)$ until we find an x that is classified as class 1. (We can, similarly, start with an example that is correctly classified as class 1 and find the adversarial x by decreasing the value of $h(x)$ until the example is classified as class 0.)

2.1.1)

To **increase** the value of $h(x)$, what update rule should we use?

- $x^{(t)} = x^{(t-1)} - \eta \nabla_x h(x^{(t-1)})$
- $x^{(t)} = x^{(t-1)} + \eta \nabla_x h(x^{(t-1)})$

Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

2.1.2)

What is $\nabla_x h(x)$? Recall that $h(x) = \sigma(\theta^T x + \theta_0)$

Hint: Use the chain rule and the following fact: $\frac{\partial}{\partial z}\sigma(z) = \sigma(z)(1 - \sigma(z))$. Assume $\theta_0 = 0$.

2.2) Simple example

Here's a simple dataset with two data points (each data point has two features and a label):

```
x1 = np.array([[ 1. , -0.1]]).T
y1 = 1

x2 = np.array([[-1. ,  0.1]]).T
y2 = 0
```

We found the following weights for a linear logistic regression model:

```
theta = np.array([[0.01, -20]]).T
```

```
theta0 = 0
```

2.2.1)

Sketch the data points and labels in 2-D vector x space, and draw the linear classifier given by (θ, θ_0).

What is the accuracy of this hypothesis on the training data?

2.2.2)

The code below implements gradient ascent/descent to find an adversarial example, starting from x1 or x2, with `eta = 0.05`.

If you specify the `input_class` to be 1 and the `target_class` to be 0, it starts with x1 (which has label 1) and then applies **gradient descent** to try to change the input in order to increase the probability of label 0.

If you specify the `input_class` to be 0 and the `target_class` to be 1, it starts with x2 (which has label 0) and then applies **gradient ascent** to try to change the input in order to increase the probability of label 1.

Run the example with `input_class=1` and `target_class=0` for 3 or 4 iterations. How is the resulting `x` classified? Is this `x` closer to x1 or x2?

```
def run():
    return compute_modified_input(dataset="vectors", input_class=1,
                                  target_class=0, iters=4)

```

Run Code | Submit | Ask for Help

You have infinitely many submissions remaining.

2.2.3)

What can we do to be sure our `x` stays as close as it can to the example we started with? Why do we want to do that, in order to create an adversarial example?

2.2.4)

What do these adversarial examples based on small perturbations to training examples tell us about the robustness of the classifier? Can you imagine a better linear classifier based on our two original training examples that would be more robust?

Checkoff 2:
Have a check-off conversation with a staff member about the lab and all the questions above.

Ask for Help | Ask for Checkoff

Your checkoff is for Section 3, 1pm-2:30pm, 34-501.
If you're in a different section, be sure to switch your section.

Food for Thought

In machine learning, techniques like DP-SGD shown below protect individual patient privacy by adding carefully controlled random noise directly to the updated gradient during training.

Algorithm 1 Differentially private SGD (Outline)

Input: Examples $\{x_1, \ldots, x_N\}$, loss function $\mathcal{L}(\theta) = \frac{1}{N}\sum_i \mathcal{L}(\theta, x_i)$. Parameters: learning rate η_t, noise scale σ, group size L, gradient norm bound C.
Initialize θ_0 randomly
for $t \in [T]$ **do**
  Take a random sample L_t with sampling probability L/N
  Compute gradient
  For each $i \in L_t$, compute $\mathbf{g}_t(x_i) \leftarrow \nabla_{\theta_t}\mathcal{L}(\theta_t, x_i)$
  Clip gradient
  $\bar{\mathbf{g}}_t(x_i) \leftarrow \mathbf{g}_t(x_i) / \max\left(1, \frac{\|\mathbf{g}_t(x_i)\|_2}{C}\right)$
  Add noise
  $\tilde{\mathbf{g}}_t \leftarrow \frac{1}{L}\left(\sum_i \bar{\mathbf{g}}_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})\right)$
  Descent
  $\theta_{t+1} \leftarrow \theta_t - \eta_t \tilde{\mathbf{g}}_t$
Output θ_T and compute the overall privacy cost (ε, δ) using a privacy accounting method.

Algorithm for Differential Privacy Stochastic Gradient Descent (DP-SGD)

DP-SGD has two main differences from standard stochastic gradient descent (SGD):

1. **Gradient clipping** to bound the influence of any single data point by clipping the gradient's L2 norm to the threshold C
2. **Adding Gaussian noise** proportional to C and σ to the averaged gradients.

Recently, in September 2025, Google Research released VaultGemma, a large language model (LLM) trained from scratch with differential privacy, which is the first of its kind at this scale. Unlike prior works that apply DP only during fine-tuning, VaultGemma integrates privacy into the entire pretraining process. It uses a modified version of DP-SGD called Scalable DP-SGD, which uses a different data-sampling method from DP-SGD, processing data in fixed-size batches by adding extra padding or trimming. With performance similar to GPT-2, VaultGemma doesn't offer the same performance as cutting-edge large language models. However, this proof of concept demonstrates that privacy-first AI models are feasible at a larger scale than was previously thought practical.

Survey

(The form below is to help us improve/calibrate for future assignments; submission is encouraged but not required. Thanks!)

How did you feel about the **length** of this lab?

- ○ Too long.
- ○ About right.
- ○ Too short.

How did you feel about the **difficulty** of this lab?

- ○ Too hard. We should tone it down.
- ○ About right.
- ○ Too easy. I want more challenge.

Do you have any feedback or comments about any questions in this lab? Anything else you want us to know?

Submit